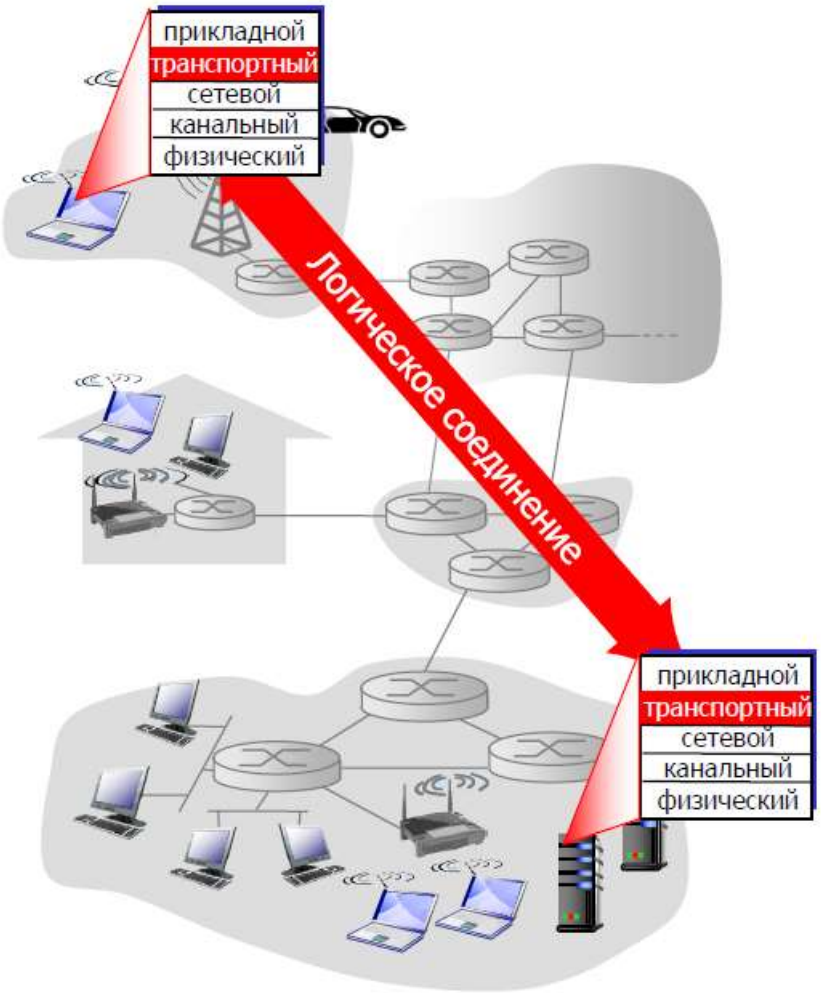
ТРАНСПОРТНЫЙ УРОВЕНЬ

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

Протоколы и службы транспортного уровня

- Предоставляют логическое соединение между процессами приложений, запущенными на разных хостах
- Транспортные протоколы действуют на конечных системах соединения
  - Сторона отправителя: разделяя сообщения приложения на сегменты, передает их на сетевой уровень
  - Сторона получателя: передает прикладному уровню сообщения, собранные из сегментов
- Для приложений доступно несколько транспортных протоколов:
  - TCP и UDP



Транспортный и сетевой уровни

- Сетевой уровень: логическое соединение между хостами
- Транспортный уровень: логическое соединение между процессами
  - Опираясь на сервисы сетевого уровня, функционально дополняет их

Аналогия: 12 детей в доме Анны отправляют письма 12 детям в доме Ивана

- хосты = дома
- процессы = дети
- Сообщения приложений = письма в конвертах
- Транспортный протокол = Анна и Иван, занимающиеся сбором и отправкой почты в своих домах
- Протокол сетевого уровня = почтовая служба

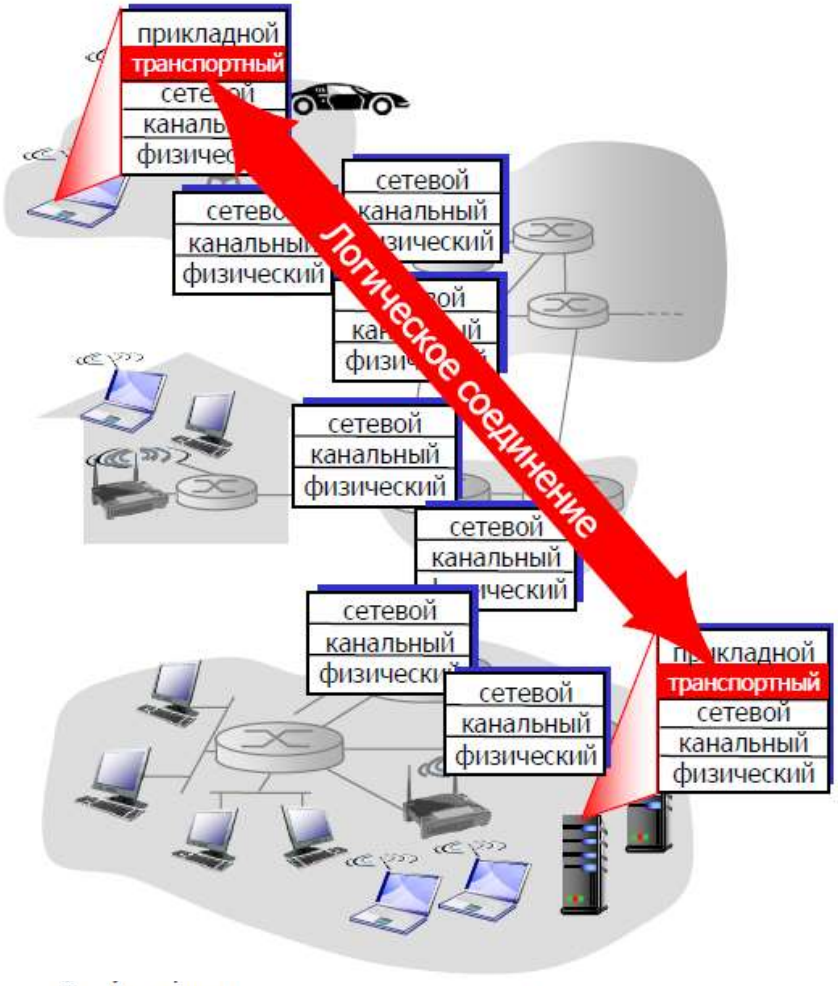
- С точки зрения детей, Анна и Иван являются почтовой службой
- Они выполняют свои обязанности **только в собственных домах**. Они не вовлечены, например, в сортировку писем в любом промежуточном почтовом отделении или их пересылку от одного почтового отделения до другого

Интернет-протоколы транспортного уровня

- Надежная, упорядоченная передача данных (протокол TCP)
  - Управление перегрузками
  - Управление потоком
  - Установление соединения

- Ненадежная, неупорядоченная передача данных (протокол UDP)
  - Простое и эффективное расширение протокола IP, работающего по принципу негарантированной доставки

- Не гарантируется ни одним из протоколов:
  - Максимальная допустимая задержка
  - Стабильная пропускная способность

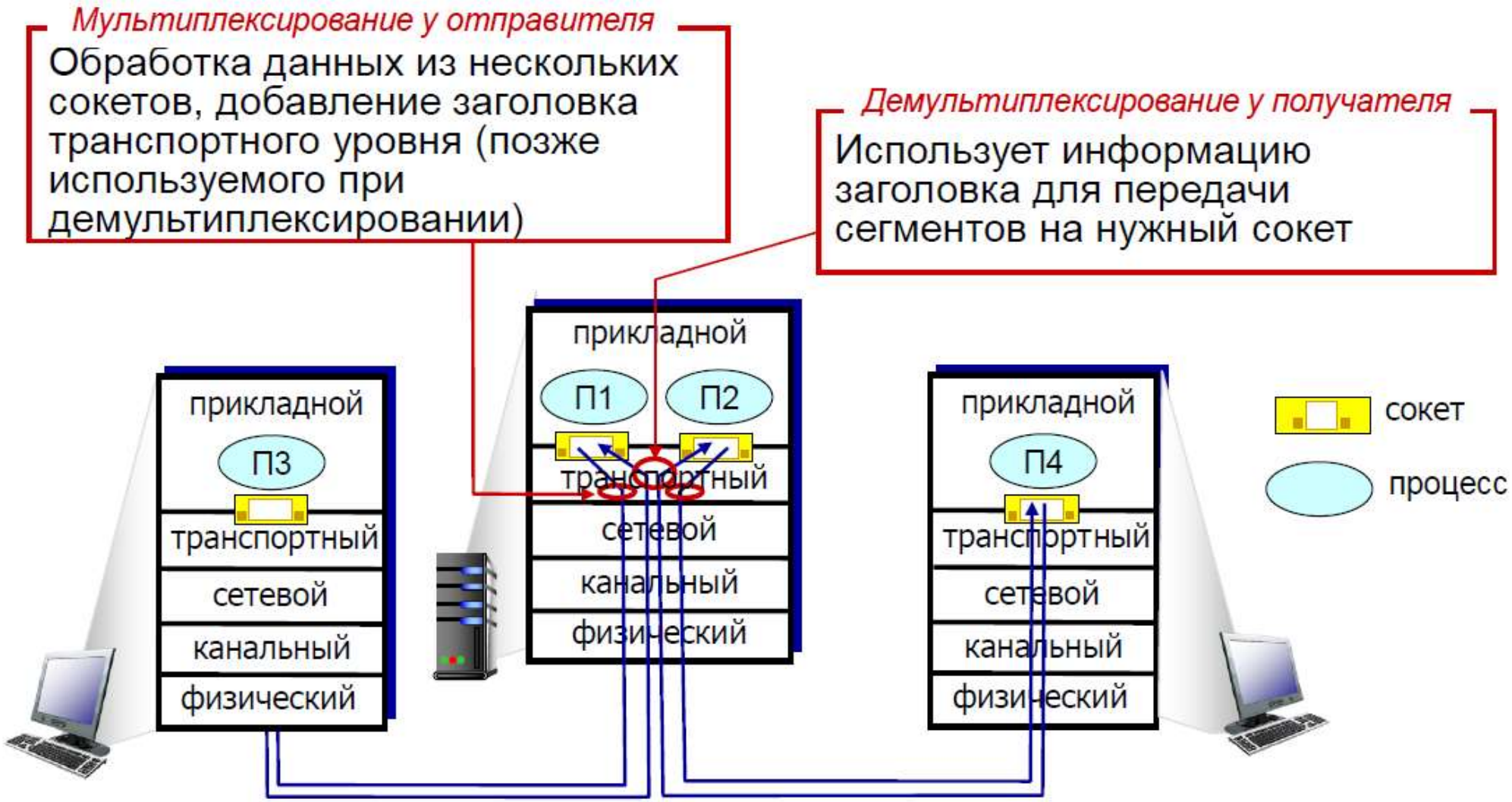


Мультиплексирование и демультиплексирование

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

Мультиплексирование / демультиплексирование



Как работает демультиплексирование

- Хост получает IP-дейтаграммы
  - Каждая дейтаграмма содержит IP-адрес отправителя и получателя
  - В каждой IP-дейтаграмме содержится один сегмент транспортного уровня
  - В каждом сегменте указаны номера портов отправителя и получателя

- Хост использует IP-адреса и номера портов для передачи сегмента на нужный сокет

- Номер каждого порта — это 16-разрядное число. Номера портов в диапазоне от 0 до 1023 называются зарезервированными

Структура сегмента TCP/UDP

Демультиплексирование без установления логического соединения: **UDP**

- Созданный сокет имеет номер порта локального хоста:

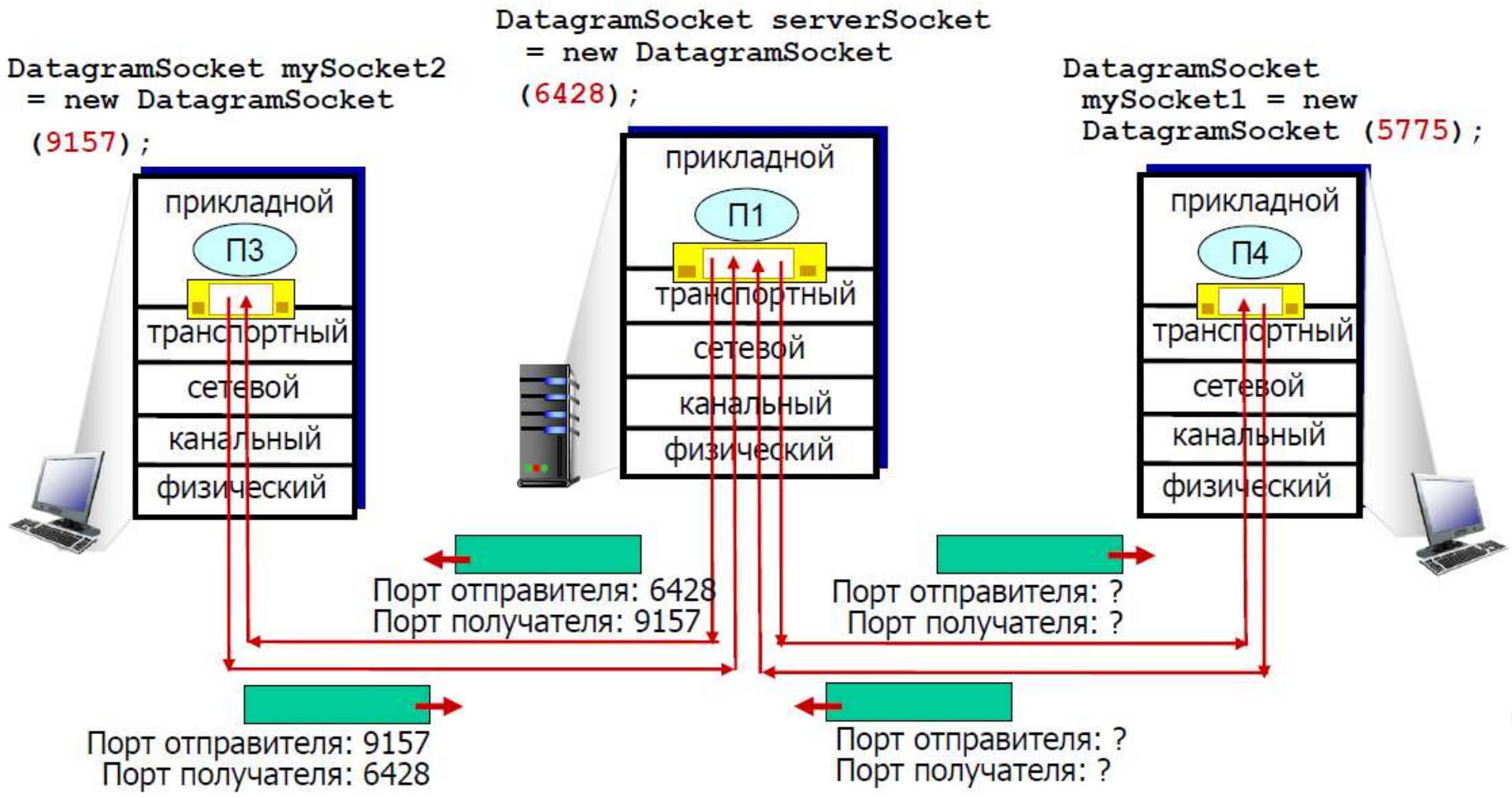
```
DatagramSocket mySocket1  = new DatagramSocket(12534);
```

- При получении хостом **UDP-сегмента**:
  - Проверка номера порта получателя в сегменте
  - UDP-сегмент направляется сокету *с таким же номером порта*

- При создании дейтаграммы для отправки UDP-сегмента необходимо определить:
  - IP-адрес получателя
  - Номер порта получателя

- **(!)** IP-дейтаграммы с одинаковыми номерами портов получателя, но с разными IP-адресами отправителей и/или номерами портов отправителя будут направлены одному и тому же сокету получателя

Демультиплексирование без установления логического соединения: пример



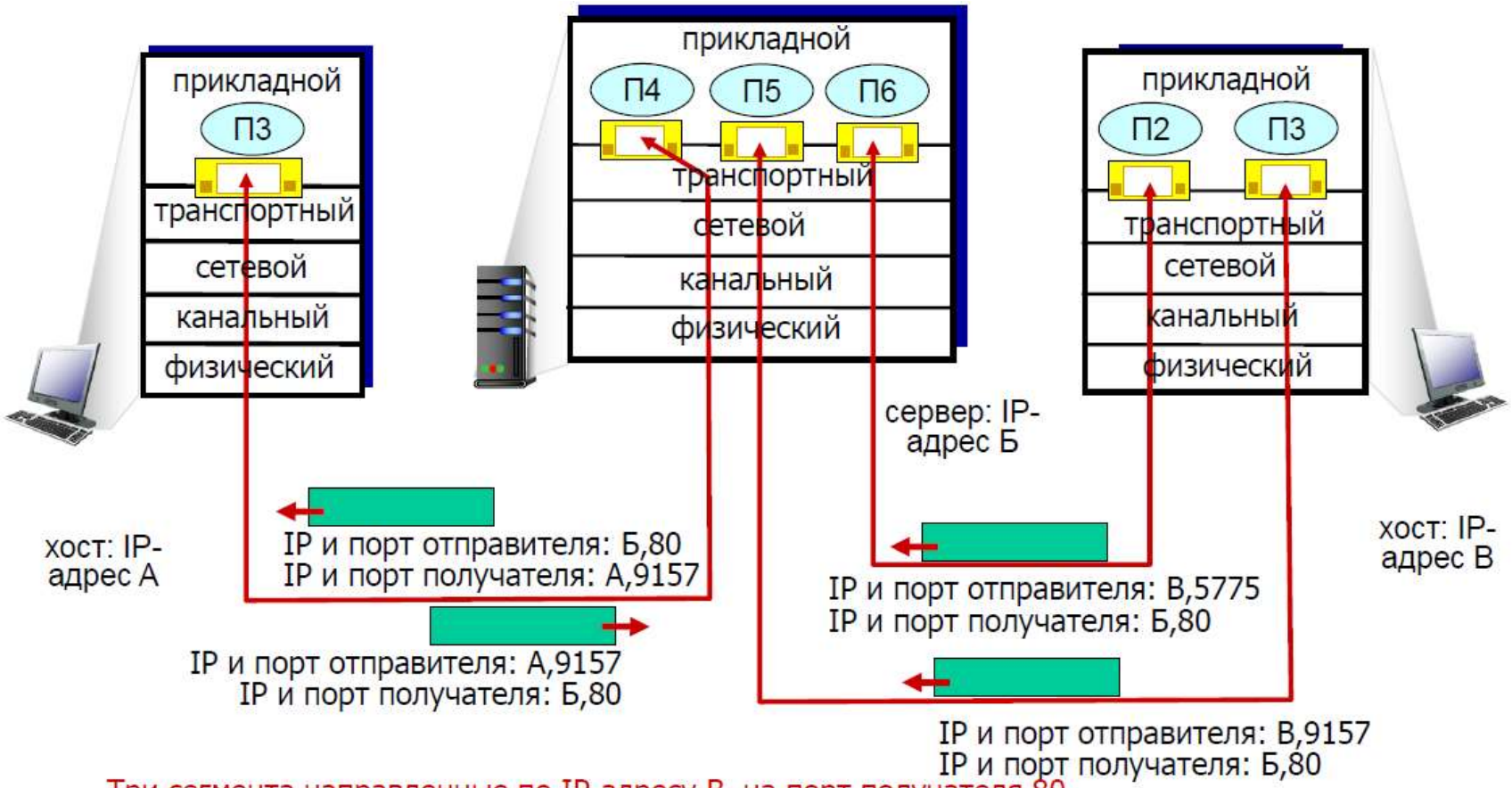
Демультиплексирование с установлением логического соединения: **TCP**

- TCP-сокет идентифицируется при помощи не двух, а **четырех** составляющих:
  - IP-адрес отправителя
  - Номер порта отправителя
  - IP-адрес получателя
  - Номер порта получателя

- Демультиплексирование: получатель использует все четыре значения для передачи сегмента на соответствующий сокет

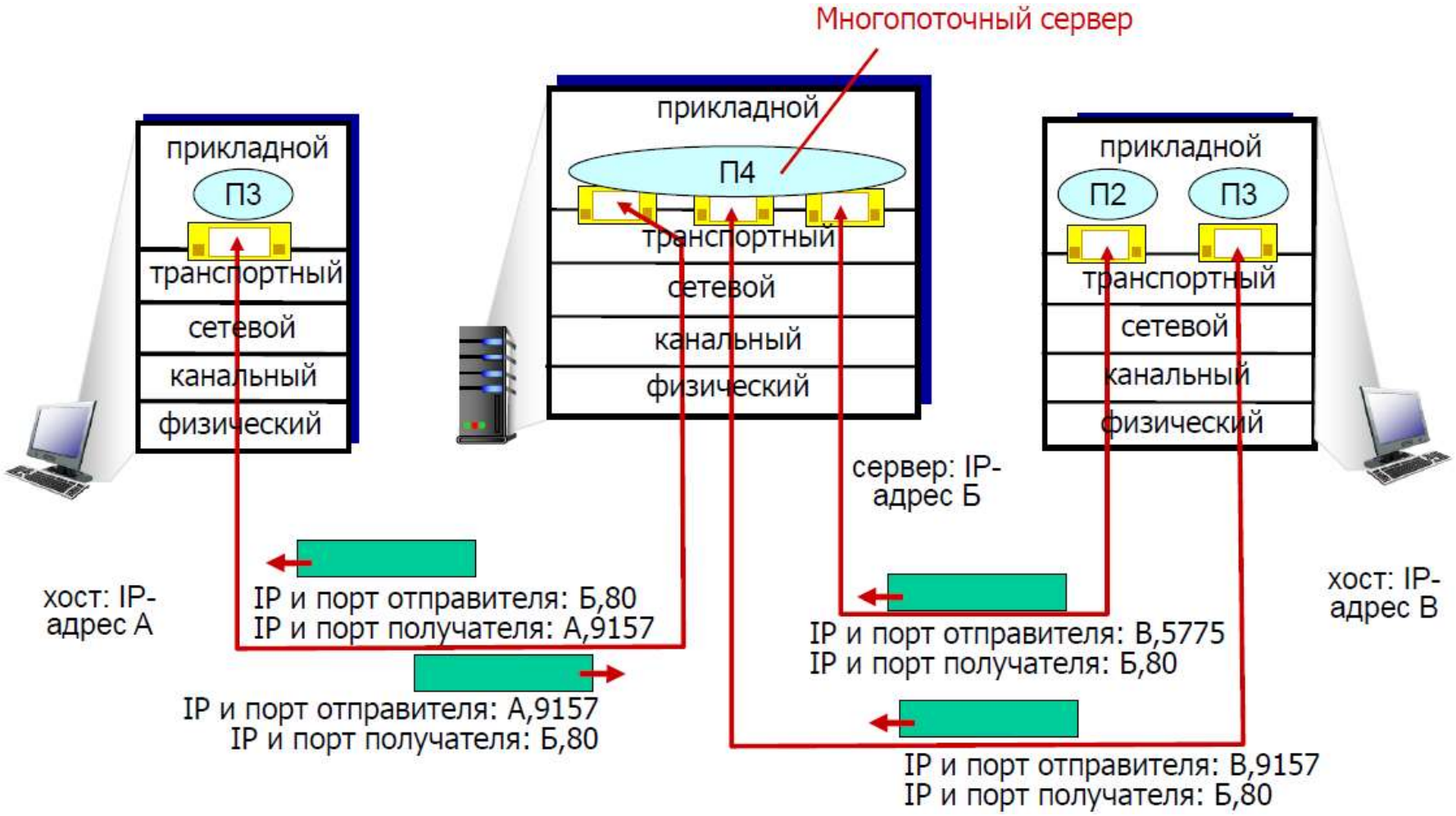
- Хост-сервер может поддерживать множество одновременных TCP-сокетов:
  - Каждый сокет определяется собственным набором значений
- Веб-серверы имеют разные сокеты для каждого клиента
  - При непостоянном соединений каждое HTTP-соединение будет иметь отдельный сокет

Демультиплексирование с установлением логического соединения: пример



Три сегмента направленные по IP-адресу В, на порт получателя 80, демультиплексированы к *разным* сокетам

Демультиплексирование с установлением логического соединения: пример



Передача данных без установления логического соединения: протокол UDP

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

UDP: Протокол пользовательских дейтаграмм [RFC 768]

- «без излишеств», «минимальный» транспортный протокол Интернета
- Служба негарантированной доставки допускает:
  - потерю UDP-сегментов
  - доставку UDP-сегментов приложению с нарушением порядка
- Без установления соединения:
  - Нет процедуры установления соединения между UDP-отправителем и получателем
  - каждый UDP-сегмент обрабатывается независимо от других

- Протокол UDP используют:
  - Потоковые мультимедийные приложения (допускающие потери и чувствительные к скорости)
  - DNS
  - Удаленный файловый сервер

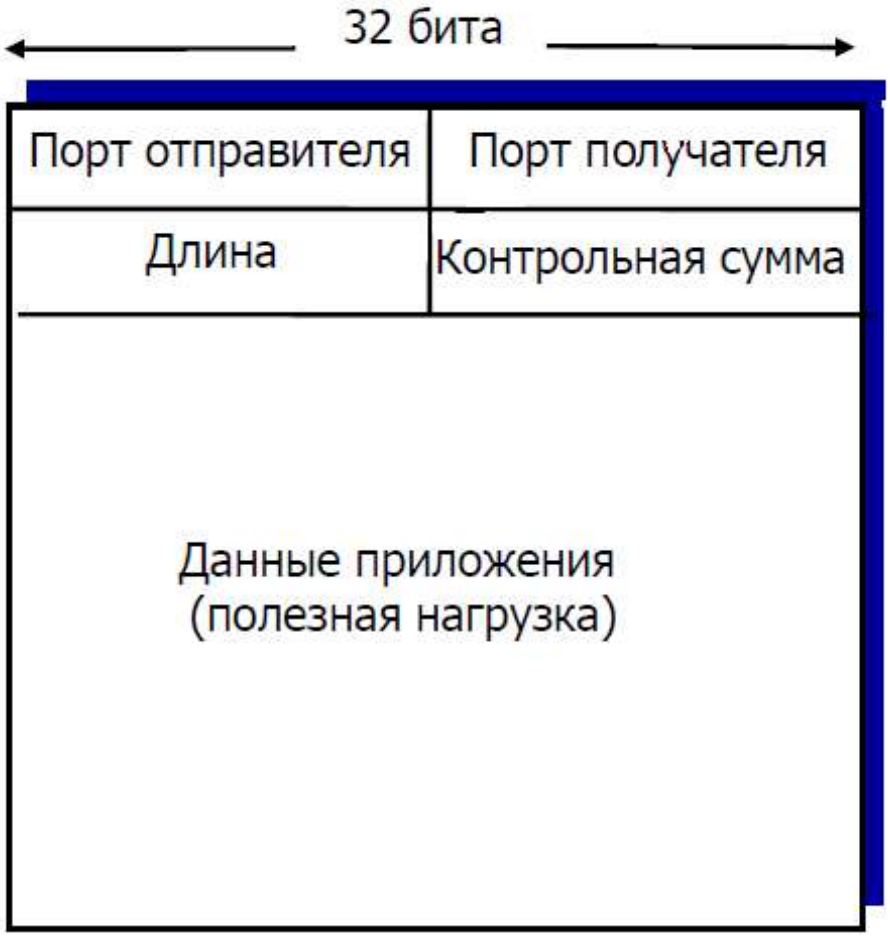
- Надежная передача данных по протоколу UDP:
  - Добавление надежной передачи на прикладном уровне
  - Анализ ошибок для конкретных приложений

Преимущества UDP перед TCP

Для чего нужен UDP? Преимущества перед TCP:

- Более полный и точный контроль приложения за процессом передачи данных
- Нет установления соединения (что может добавлять задержку)
- Простота: состояния соединения не сохраняются у получателя и отправителя
- Малый размер заголовка (8 байт, в TCP - 20 байт)
- Нет контроля перегрузки: UDP может действовать так быстро, как пожелает

UDP: заголовок сегмента



Структура UDP-сегмента

Контрольная сумма UDP

Цель: обнаружение «ошибок» (например, инвертированные разряды) в переданном сегменте

Отправитель:

- Представление содержимого сегмента, включая поля заголовка в виде последовательности 16-битных целых чисел
- Контрольная сумма: дополнение к содержимому сегмента (дополнение суммы до единицы)
- Отправитель помещает значение контрольной суммы в поле **контрольная сумма** UDP-сегмента

Получатель:

- вычисление контрольной суммы полученного сегмента
- соответствует ли полученное значение значению в поле контрольная сумма:
  - NO – обнаружена ошибка
  - YES – ошибок не обнаружено

Пример расчета контрольной суммы

Есть три следующих 16-битных слова:
0110011001100000
0101010101010101
1000111100001100

Сумма первых двух слов равна:
1011101110110101

Сложение полученной суммы с третьим словом дает:
0100101011000001

Дополнение до единицы производится заменой всех 0 на 1
Для 0100101011000001 это:
1011010100111110
(это и есть контрольная сумма)

На стороне получателя складываются все четыре слова: три слова сегмента и контрольная сумма. Сумма без ошибок должна быть равна:
1111111111111111

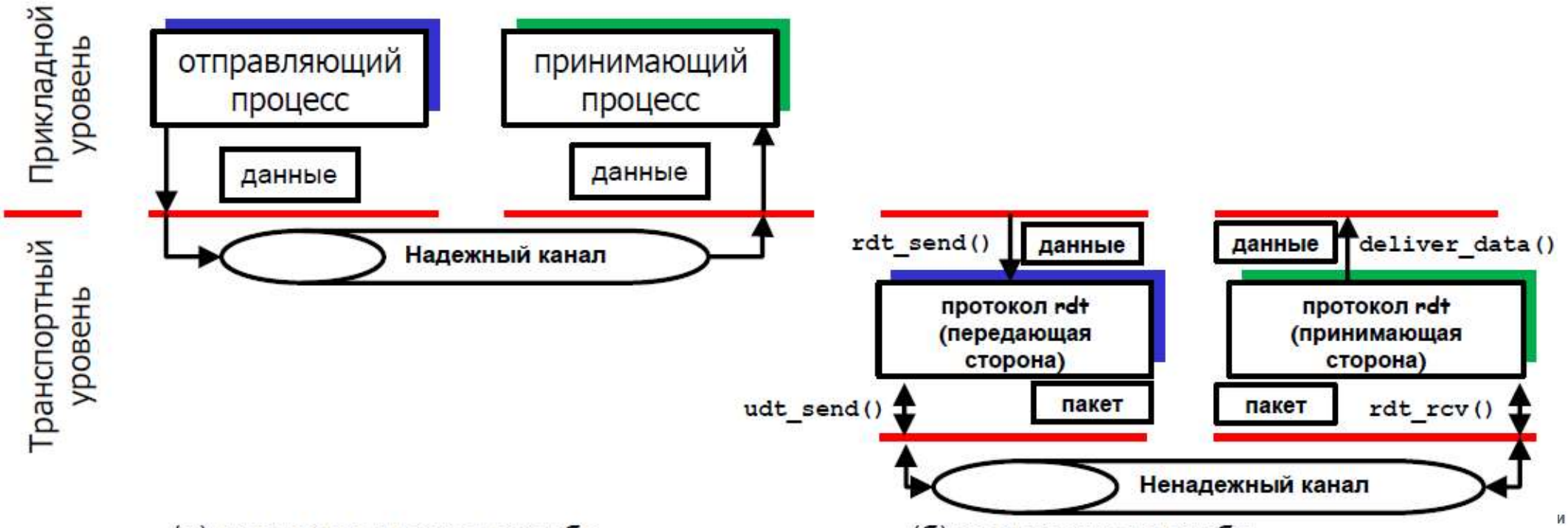
Принципы надежной передачи данных

Санкт-Петербургский государственный университет

Факультет математики и компьютерных наук

Принципы надежной передачи данных

- Важно для прикладного, транспортного и канального уровней
- Характеристики ненадежного канала будут полностью определять сложность протокола надежной передачи данных (reliable data transfer - rdt)



(а) предоставляемая служба

(б) реализация службы

Надежная доставка данных: начало

rdt_send(): вызывается сверху, например, приложением. Передает данные вышестоящему уровню получателя

deliver_data(): вызывается протоколом rdt для передачи данных наверх

udt_send(): вызывается протоколом rdt для передачи данных получателю по ненадежному каналу

rdt_rcv(): вызывается при передаче пакета получателю из канала

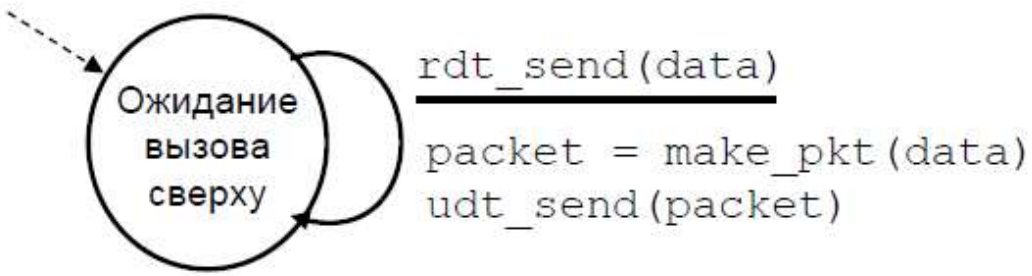
Надежная передача данных: начало

Наши действия:

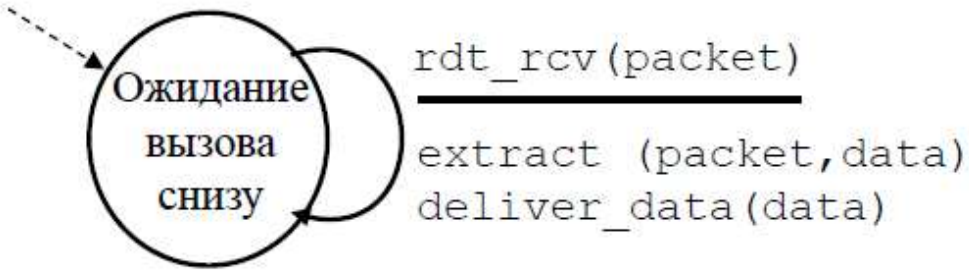
- Постепенно разработаем стороны отправителя и получателя протокола надежной передачи данных (rdt)
- Рассмотрим только однонаправленную передачу данных
- Но управляющая соединением информация будет распространяться в обе стороны!
- Для описания отправителя и получателя используем FSM-схемы (finite-state machine)

Протокол rdt1.0: передача данных по надежному каналу

- Нижерасположенный канал абсолютно надежен
  - Нет битовых ошибок
  - Нет потерь пакетов
- Отдельные FSM-схемы отправителя и получателя:
  - Отправитель отправляет данные в нижерасположенный канал
  - Получатель считывает данные из нижерасположенного канала



отправитель



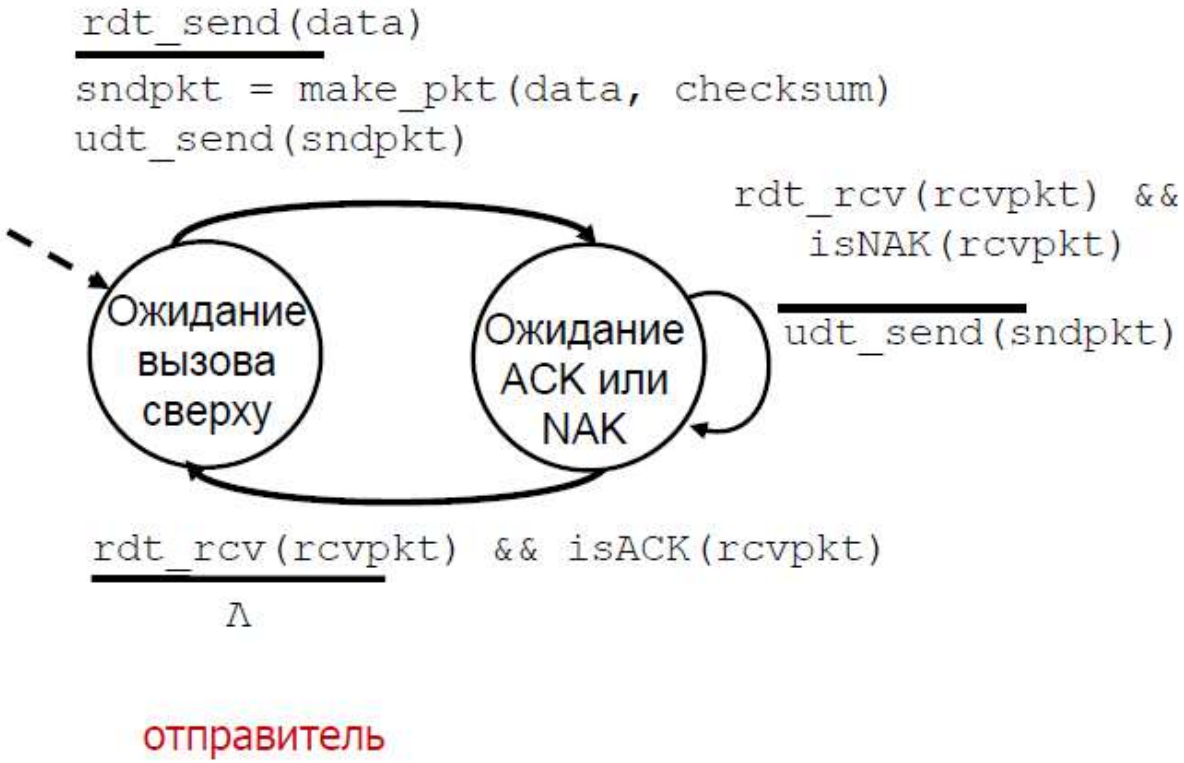
получатель

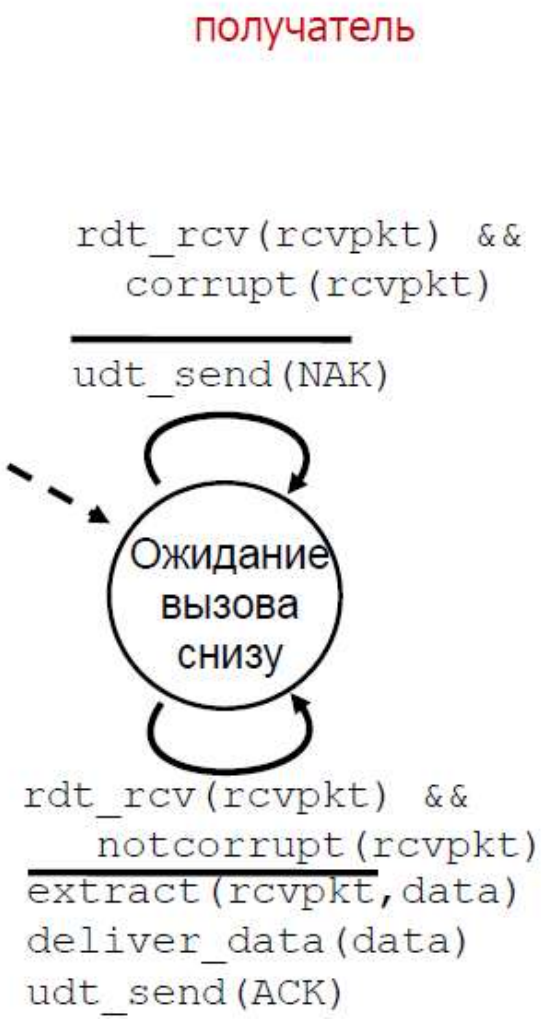
Протокол rdt2.0: канал с ошибками бит

- Ниже расположенный канал может искажать биты в пакете, но пакеты не теряются
- Контрольная сумма для обнаружения ошибок бит
- Вопрос: как исправить ошибки?
  - Подтверждения (ACK-пакеты): получатель явно сообщает отправителю, что пакет получен корректно
  - Отрицательные подтверждения (NAK-пакеты): получатель явно сообщает отправителю, что в пакете есть ошибки
  - Отправитель повторно передает пакет данных при получении NAK-пакетов

- Новые механизмы в rdt2.0 (отличие от rdt1.0):
  - Обнаружение ошибок
  - Подтверждения (ACK-пакеты и NAK-пакеты), как ответ получателя отправителю
  - Повторная передача

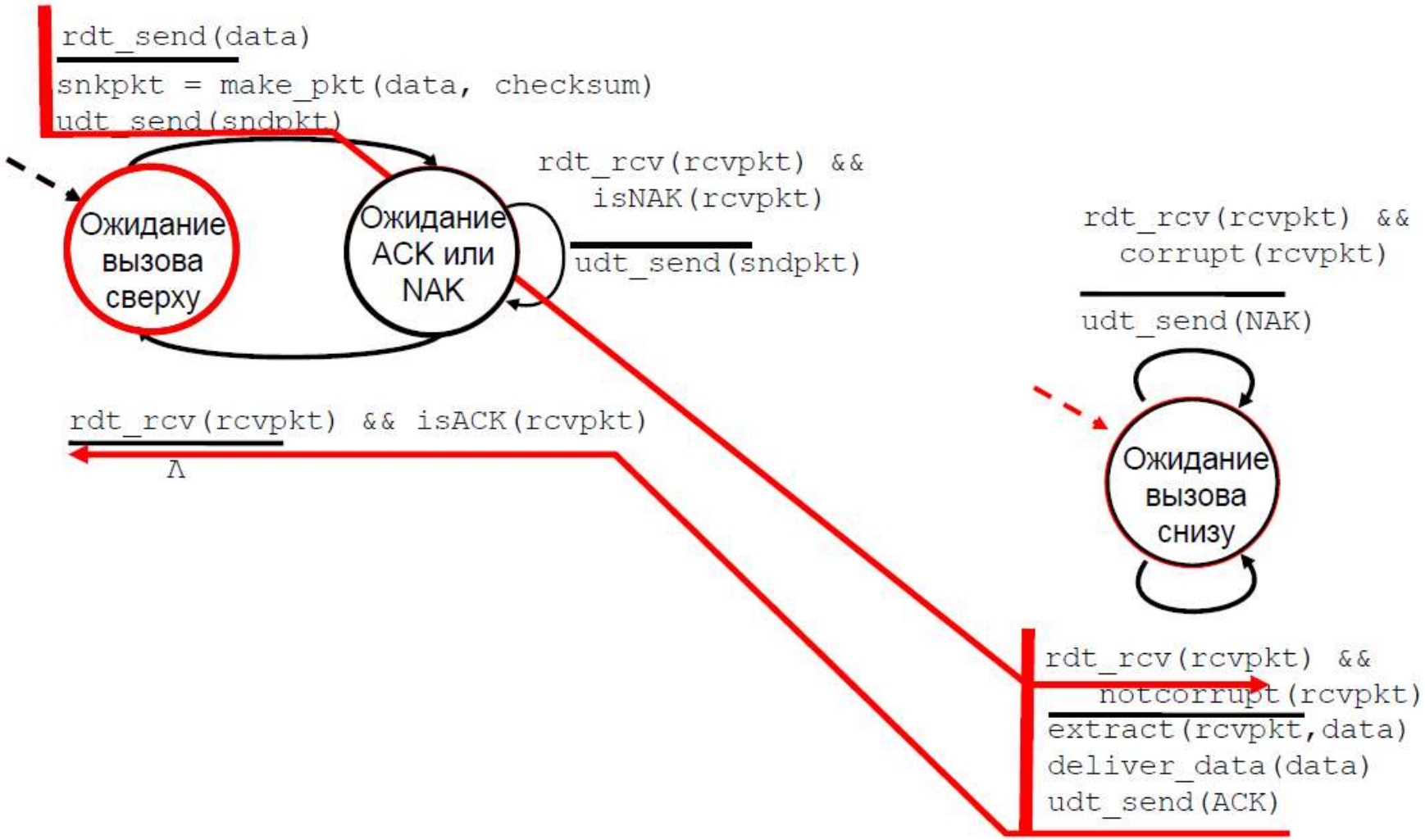
Протокол rdt2.0: FSM-схема



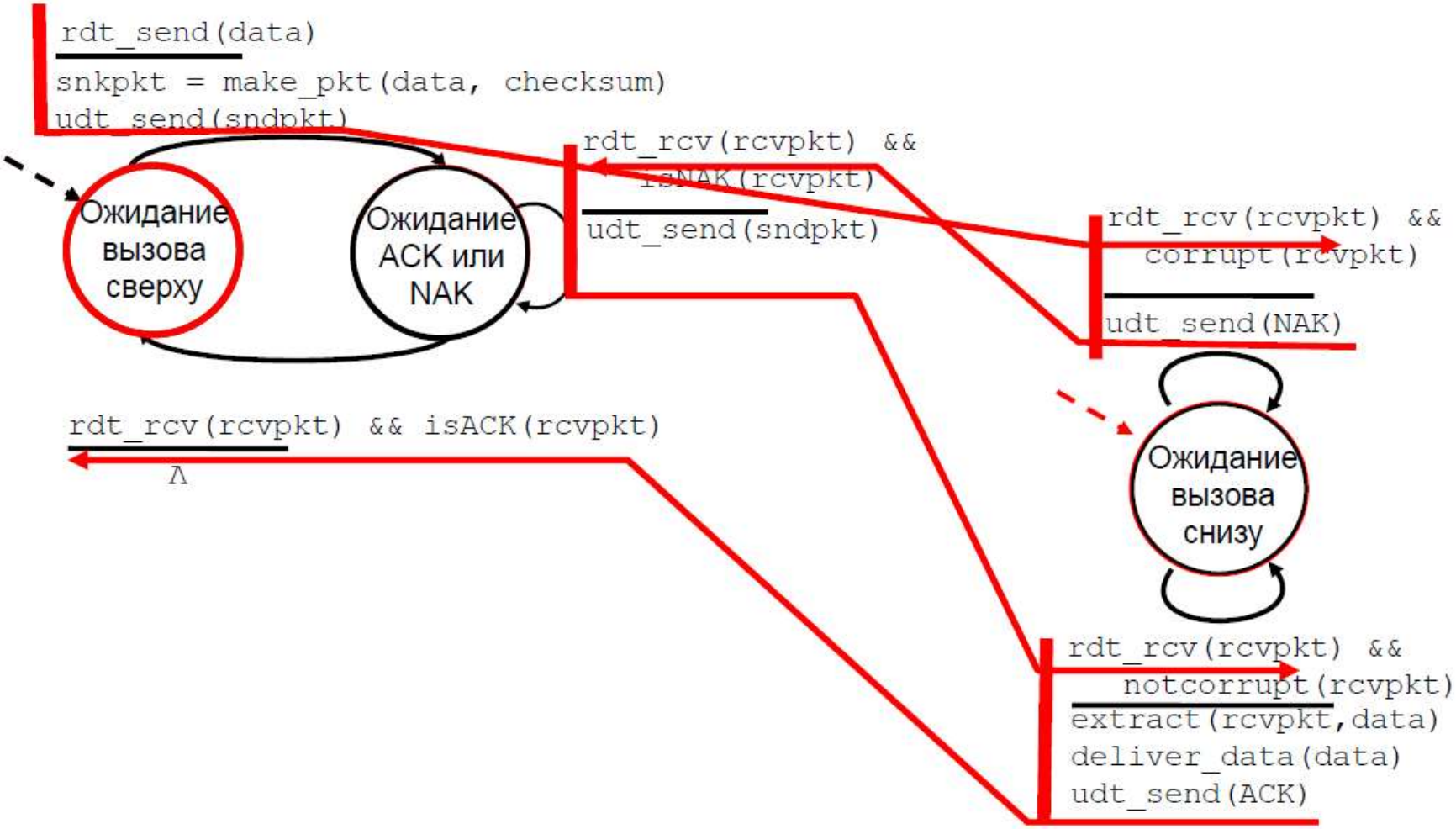


Протокол с ожиданием подтверждений (stop-and-wait).

Протокол rdt2.0: сценарий при отсутствии ошибок



Протокол rdt2.0: сценарий при наличии ошибок



Протокол rdt2.0: имеет серьезный недостаток!

Что произойдет, если ACK/NAK-пакеты повреждены?

- Отправитель не знает, что произошло у получателя
- Нет возможности просто повторить отправку, так как возможно дублирование пакетов

Существует три способа обработки поврежденных ACK или NAK:

- Отправитель повторно передает текущий пакет, если ACK/NAK-пакеты повреждены
- Отправитель добавляет контрольные биты каждому пакету
- Повторная отправка пакетов, поврежденные квитанции считаются отрицательными. Возникает проблема дублирования пакетов

Новый протокол rdt2.1: описание

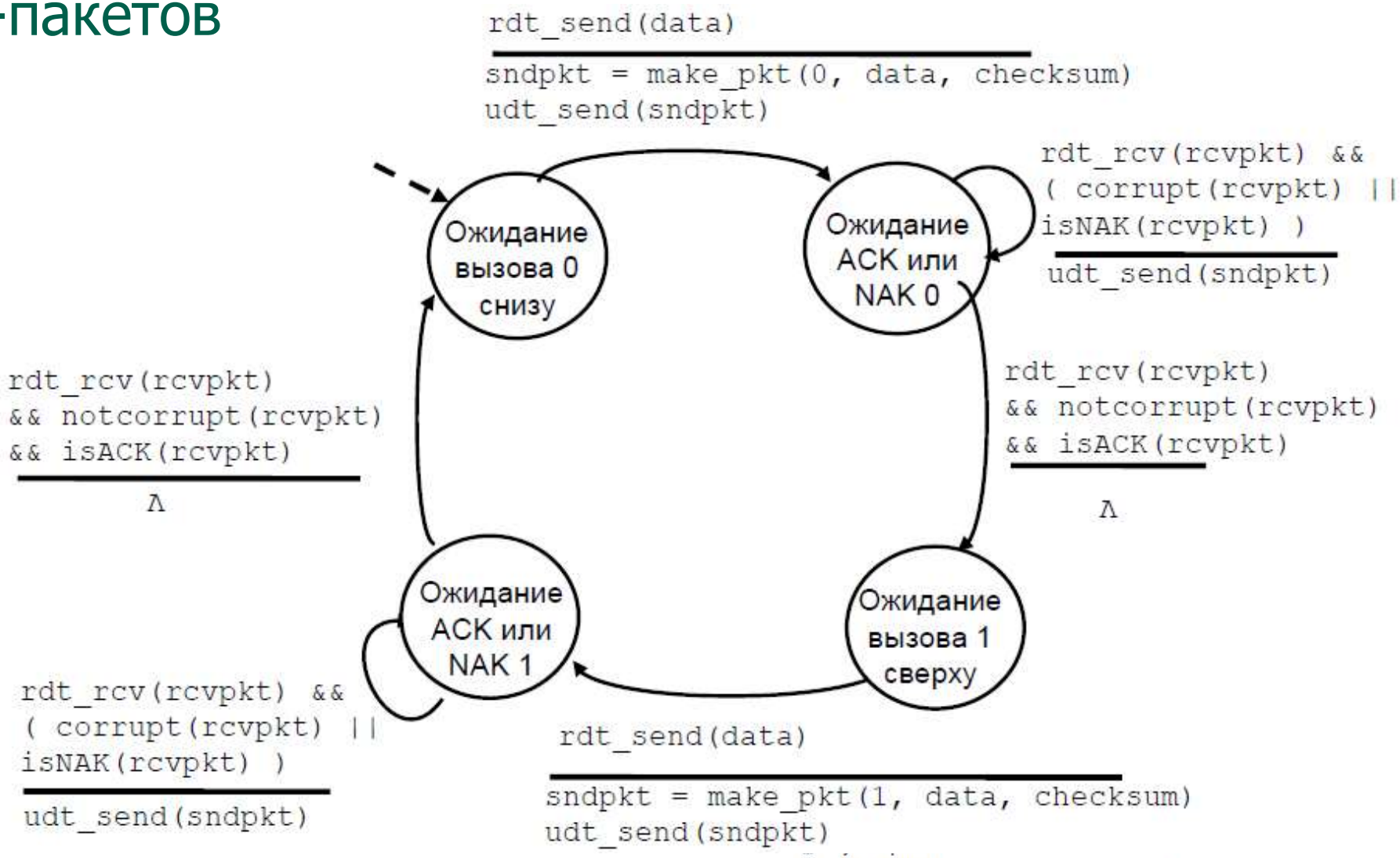
Отправитель:

- К пакету добавляется порядковый номер
- Двух порядковых номеров (0,1) будет достаточно
- Необходима проверка, если получен искаженный ACK/NAK-пакет
- В два раза больше состояний
  - Состояния должны «запоминать», какой порядковый номер должен быть у ожидаемого пакета: 0 или 1

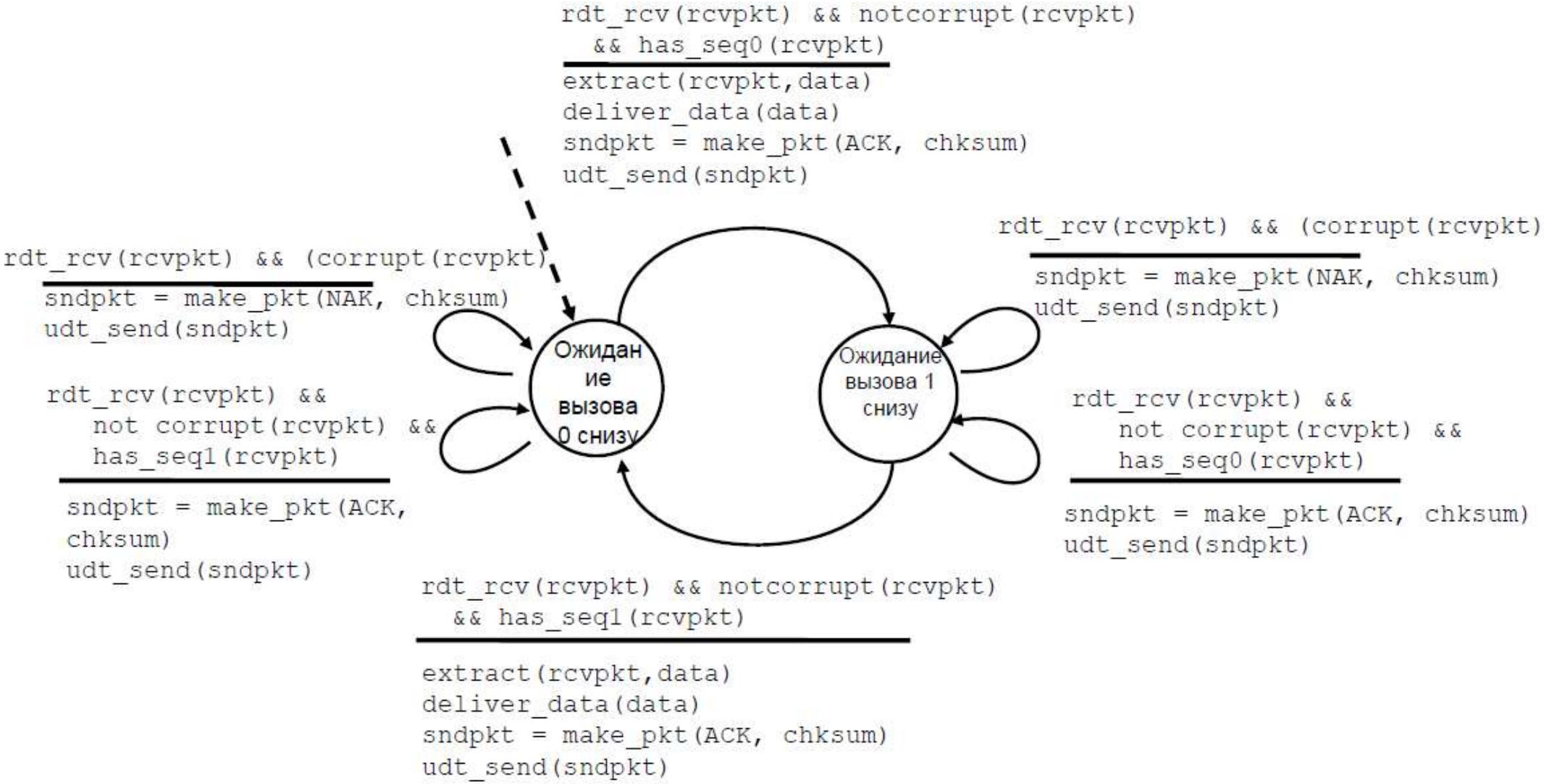
Получатель:

- Должен выполнять проверку, если полученный пакет дублируется
  - Состояние означает ожидание либо 0, либо 1 в порядковом номере пакета
- Примечание: получатель не может знать, были ли корректно получены его последние ACK/NAK-пакеты отправителем

Протокол rdt2.1: FSM-схема **отправителя**, обработка ACK/NAK-пакетов



Протокол rdt2.1: FSM-схема **получателя**, обработка ACK/NAK-пакетов



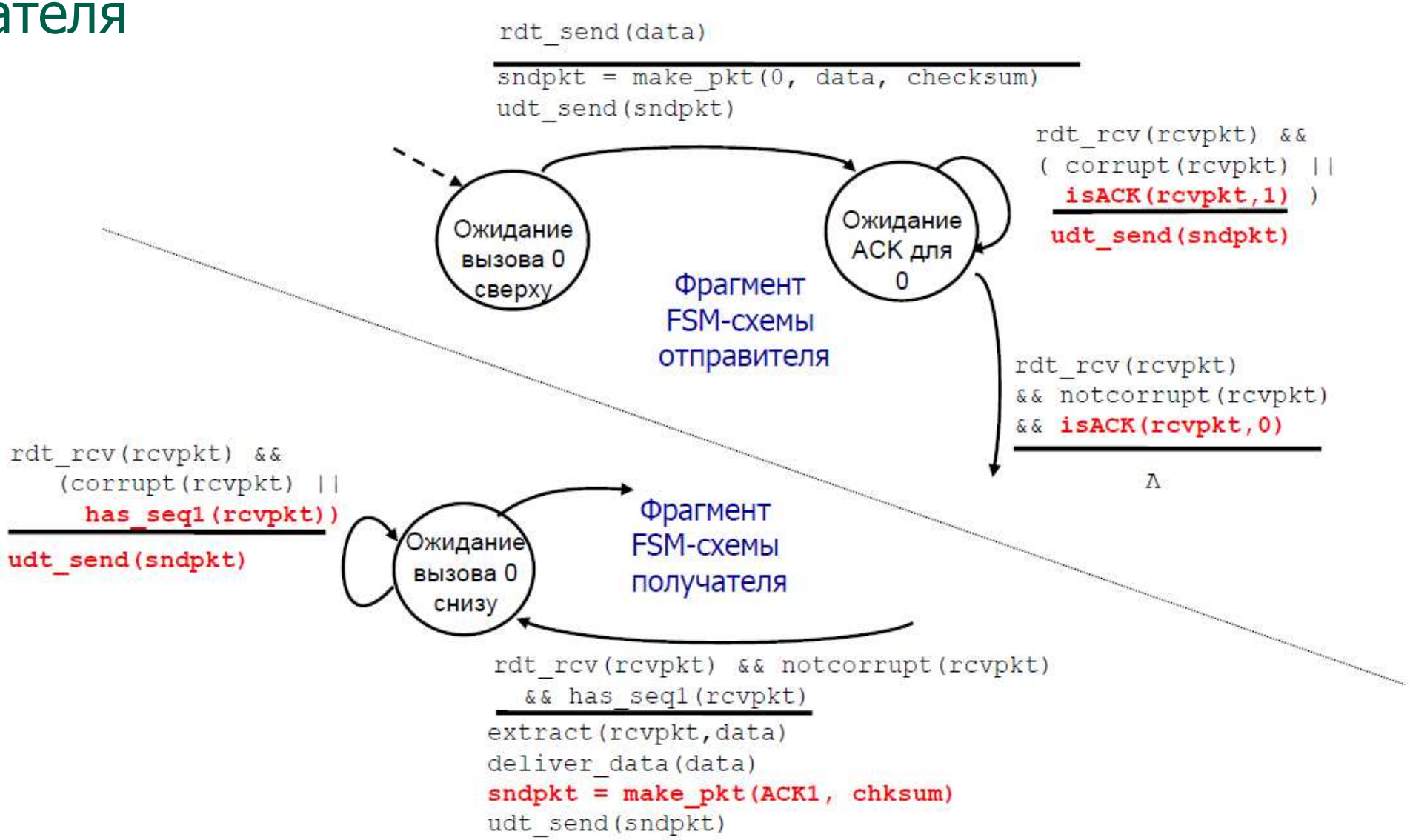
Протокол rdt2.2: протокол без NAK-пакетов

- Тот же функционал, что и у протокола rdt2.1, использующего только ACK-пакеты

- Вместо NAK-пакета получатель отправляет ACK-пакет на последний верно полученный пакет данных
  - Получатель должен явно указывать порядковый номер пакета, на который отправляет подтверждение

- Отправитель реагирует на дублирование ACK-пакета как на получение NAK-пакета: *повторяет передачу текущего пакета*
  - Если передающая сторона получит две положительные квитанции для одного и того же пакета, то это будет указывать на наличие ошибок в пакете, следующем за тем, для которого были получены сдвоенные квитанции

Протокол rdt2.2: фрагменты FSM-схем отправителя и получателя



Протокол **rdt3.0**: передача данных по каналу с возможными ошибками и потерями

Новое допущение: нижерасположенный канал может терять пакеты данных и ACK-пакеты

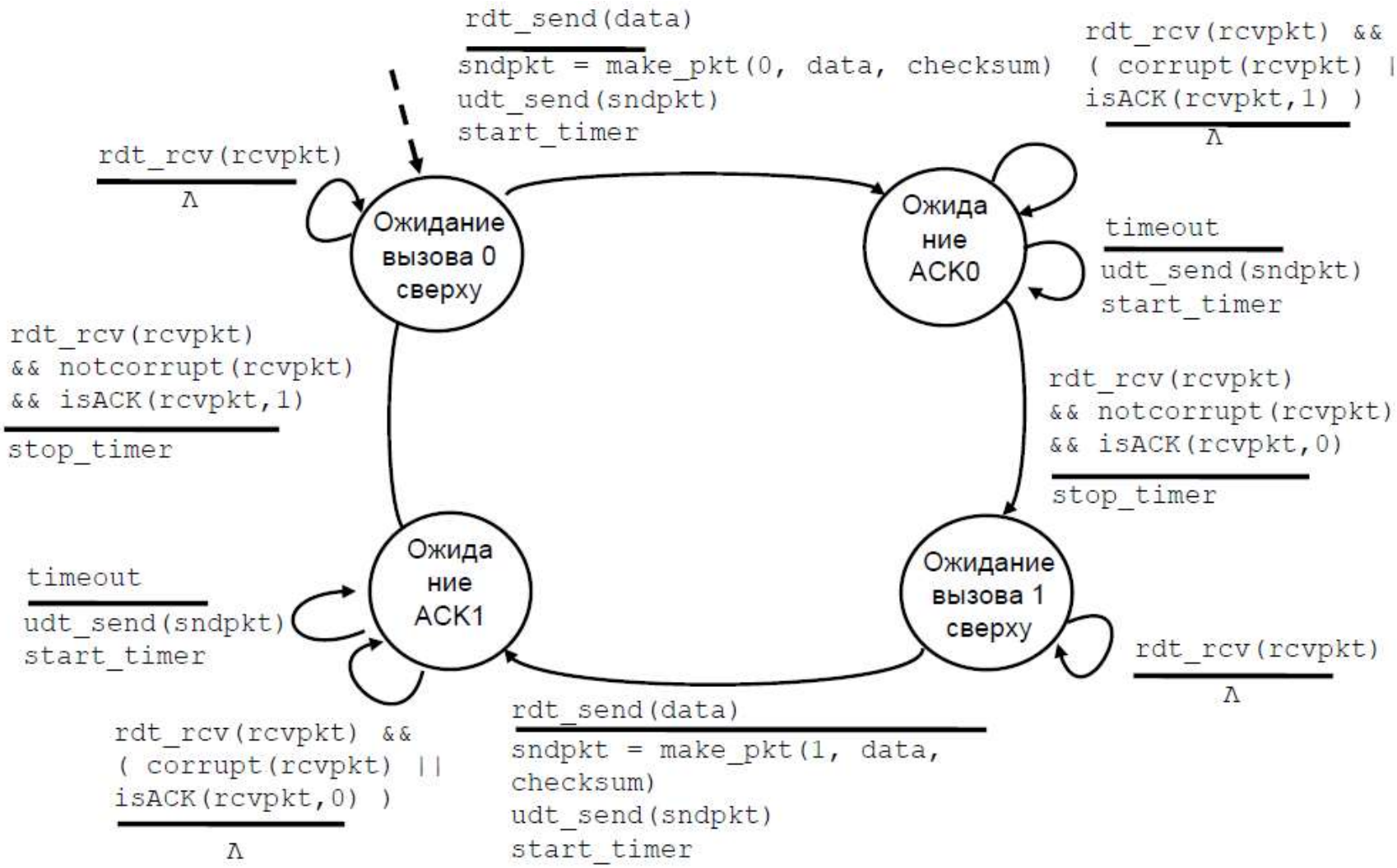
- Контрольная сумма, порядковые номера, ACK-пакеты, механизм повторных передач становятся недостаточны, хотя и могут быть полезны

Такой механизм является панацеей для передающей стороны: ей не нужно знать о том, потерян ли пакет, потеряна ли его квитанция или ничего не потеряно, но квитанция пришла с задержкой.
Во всех трех случаях производится одно и то же действие: **повторная передача пакета**

Подход: отправитель ожидает ACK-пакет в течение «приемлемого» времени

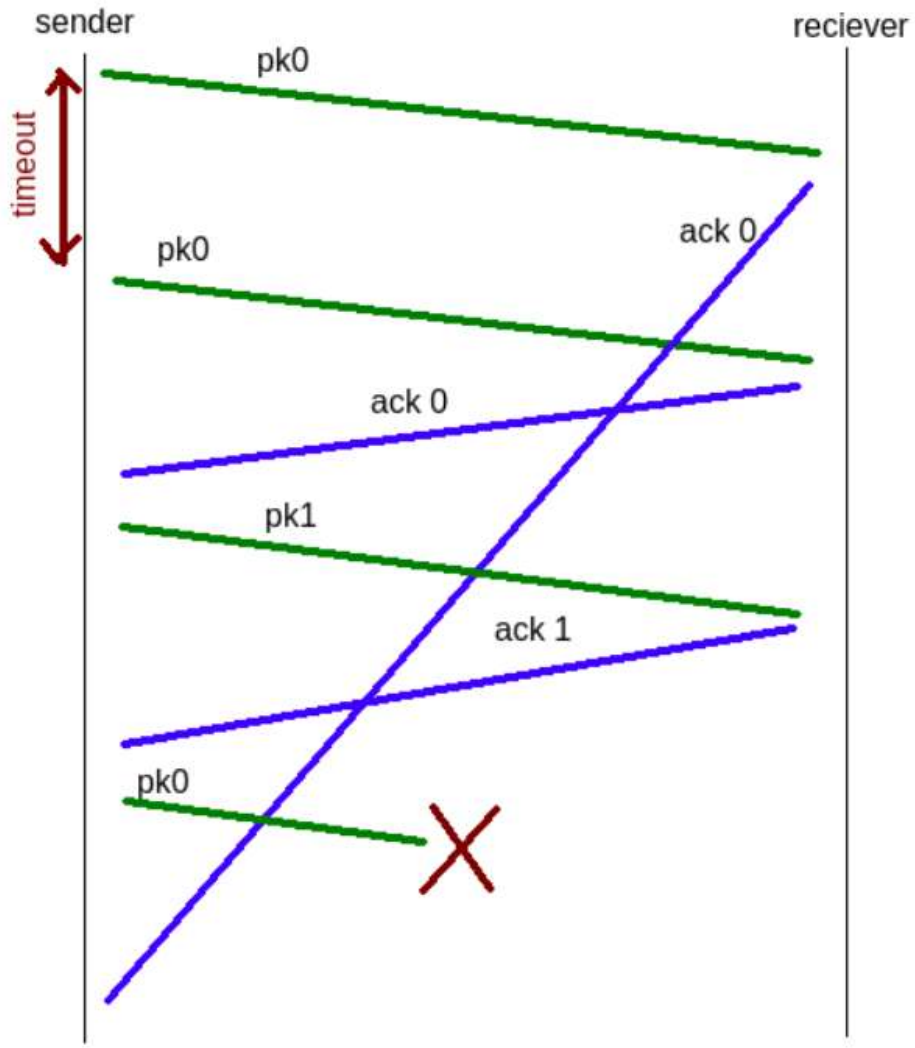
- Выполняет повторную передачу, если ACK-пакет не был получен за это время
- Если пакет данных (или ACK-пакет) только задержался, но не был потерян:
  - Повторная передача приведет к *дублированию*, но порядковый номер покажет, что пакет уже был обработан
  - Получатель должен определить порядковый номер пакета, который уже был подтвержден
- Необходим таймер для обратного отсчета времени

Протокол rdt3.0: FSM-схема отправителя

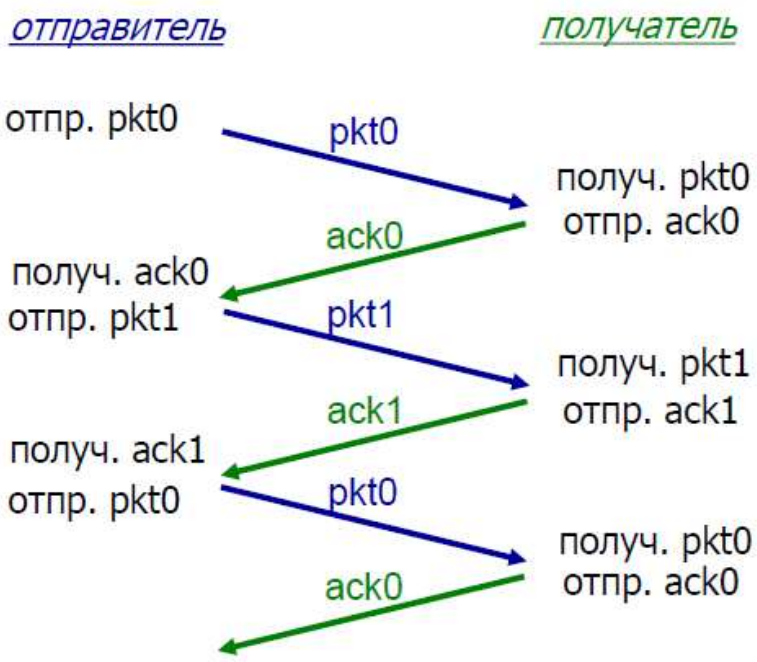


Получатель и нумерация пакетов

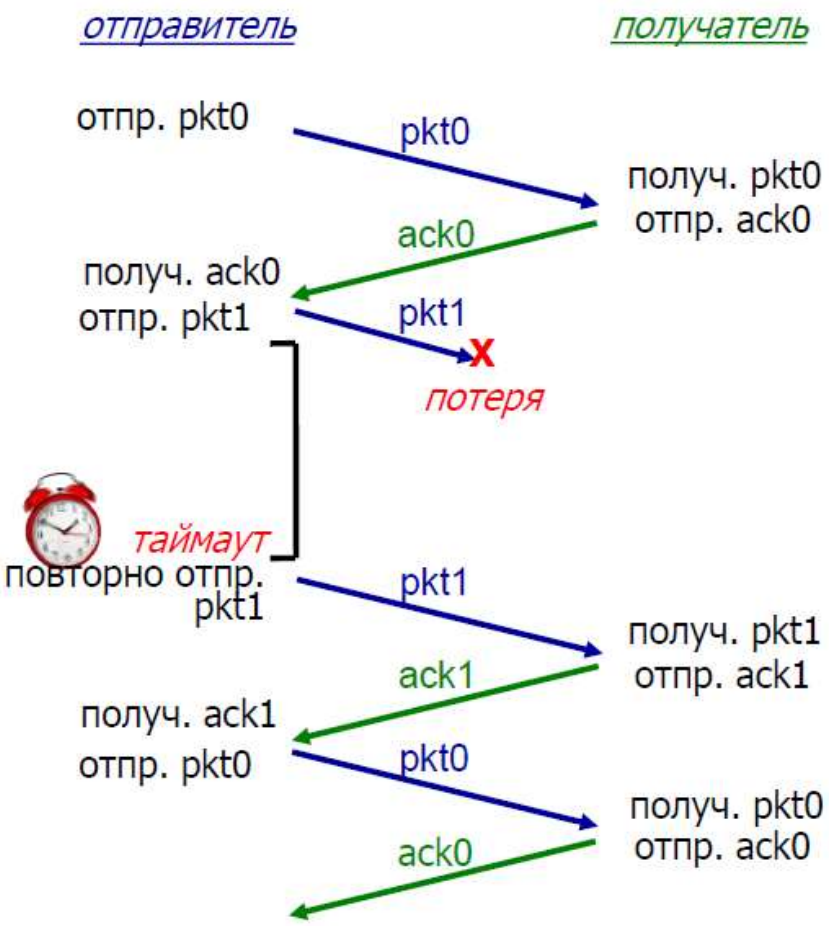
Почему нужна нумерация ACK пакетов – чтобы не сработало ложное уведомление (задержавшееся от другого пакета) и отправитель не посчитал, что пакет был доставлен корректно



Протокол rdt3.0 в действии



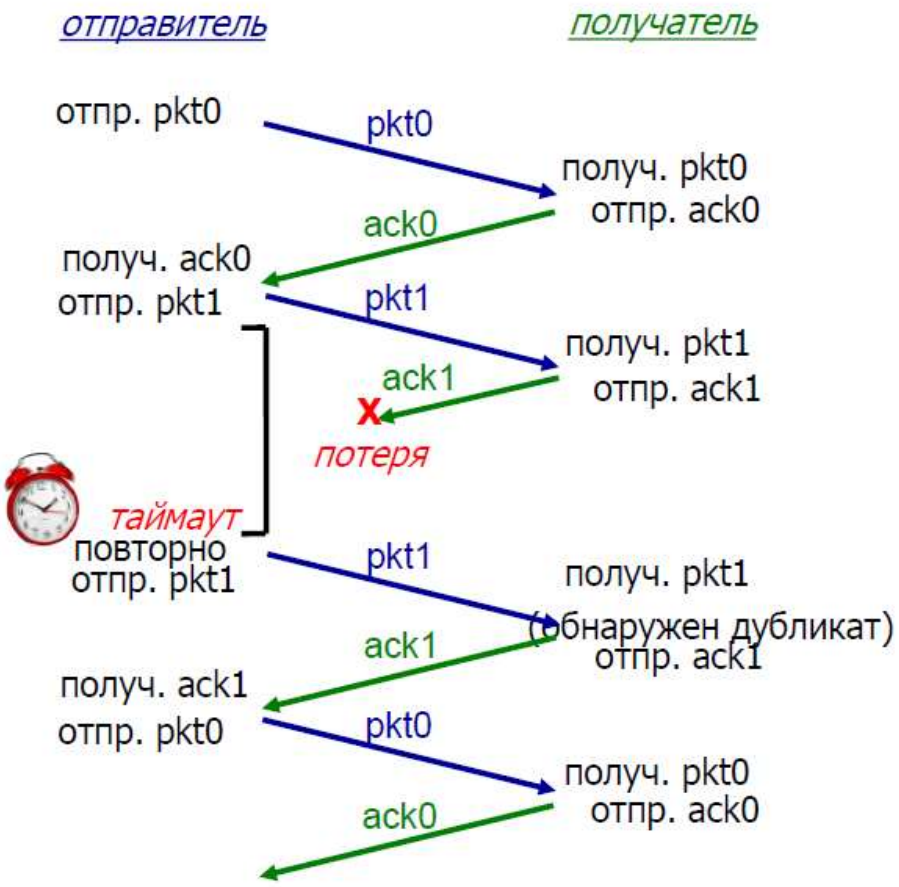
(а) Нет потерь



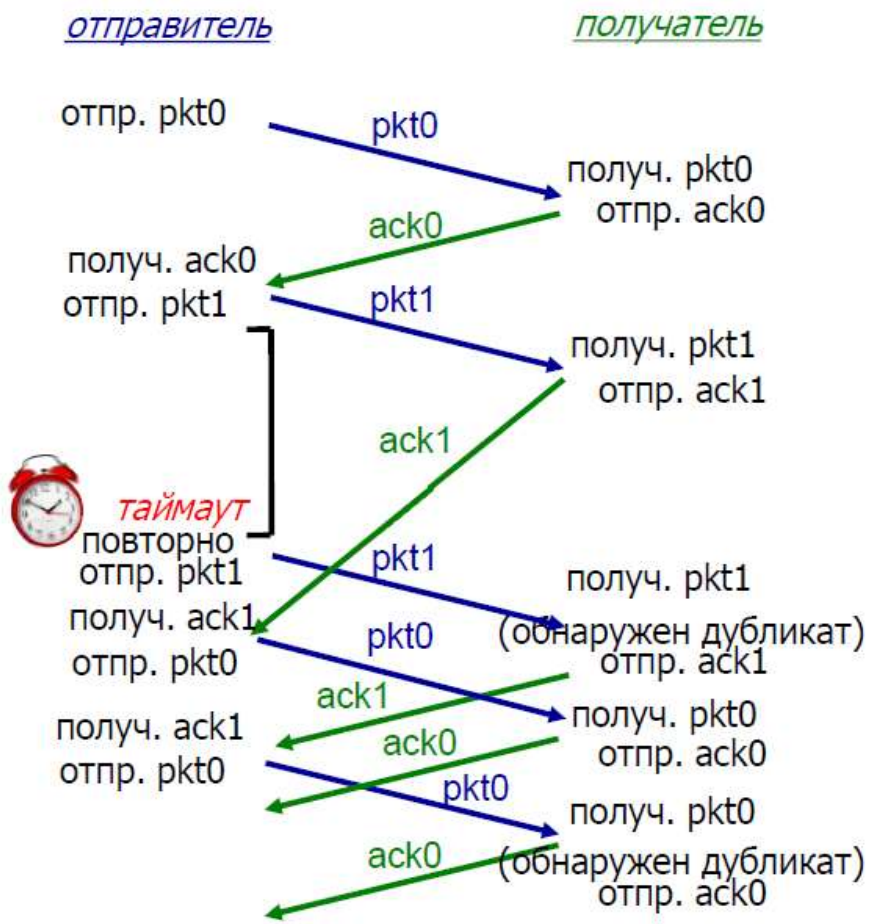
(б) Потеря пакета

Факультет математики и компьютерных наук

Протокол rdt3.0 в действии



(в) Потеря ACK



(г) Преждевременный таймаут/ задержка ACK

Производительность протокола rdt3.0

- Протокол rdt3.0 корректный, но **производительность его низкая**
- Например: канал 1 Гбит/с, размер пакета 1 Кб:

$$D_{\text{передача}} = \frac{L}{R} = \frac{8000\ \text{бит}}{10^9\text{бит/с}} = 8\ \text{мкс}$$

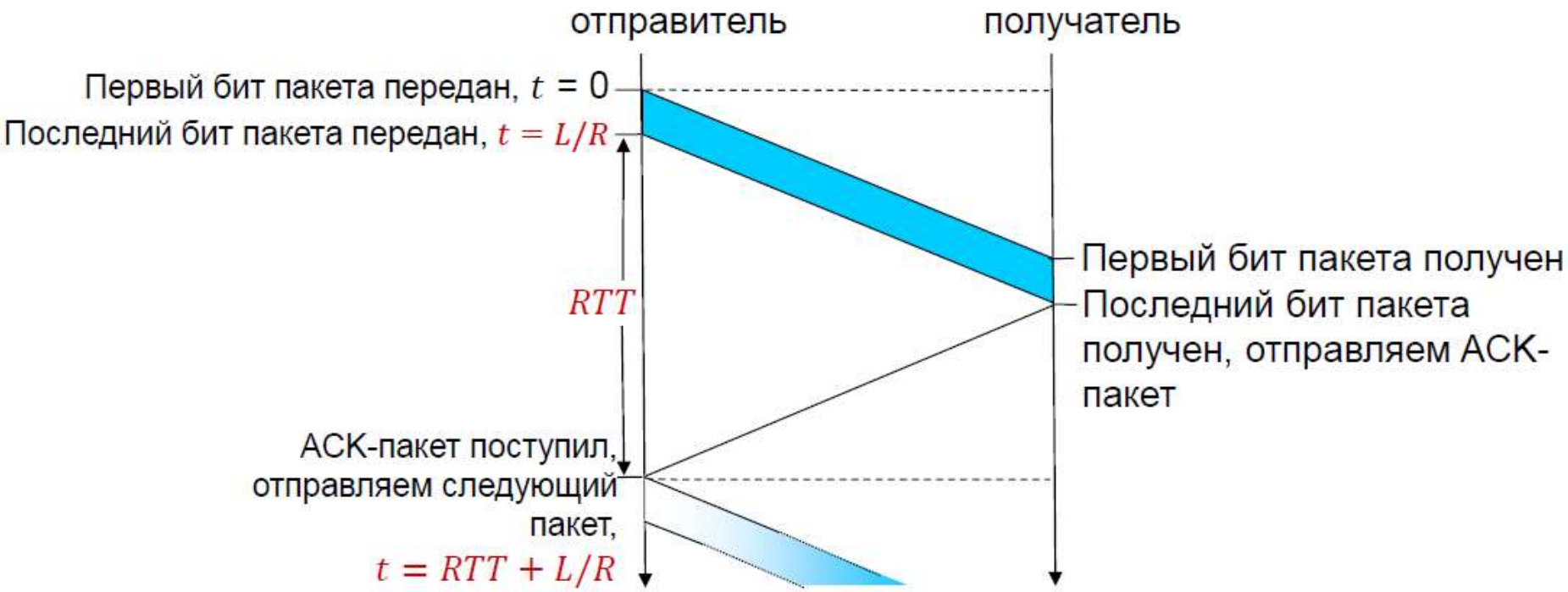
- $U_{\text{отправитель}}$ часть времени, которую отправитель занят отправкой (КПД отправителя)

$$U_{\text{отправитель}} = \frac{L/R}{RTT + L/R} = \frac{0{,}008}{30{,}008} = 0{,}00027$$

- если RTT=30 мс и пакет 1 Кб отправляется каждые 30 мс, то пропускная способность 1 Гбит/с канала равна 267 Кбит/с (= 8000 бит / 30,008 мс)

- **Сетевой протокол ограничивает использование физических ресурсов!**

Протокол rdt3.0: алгоритм с ожиданием подтверждений

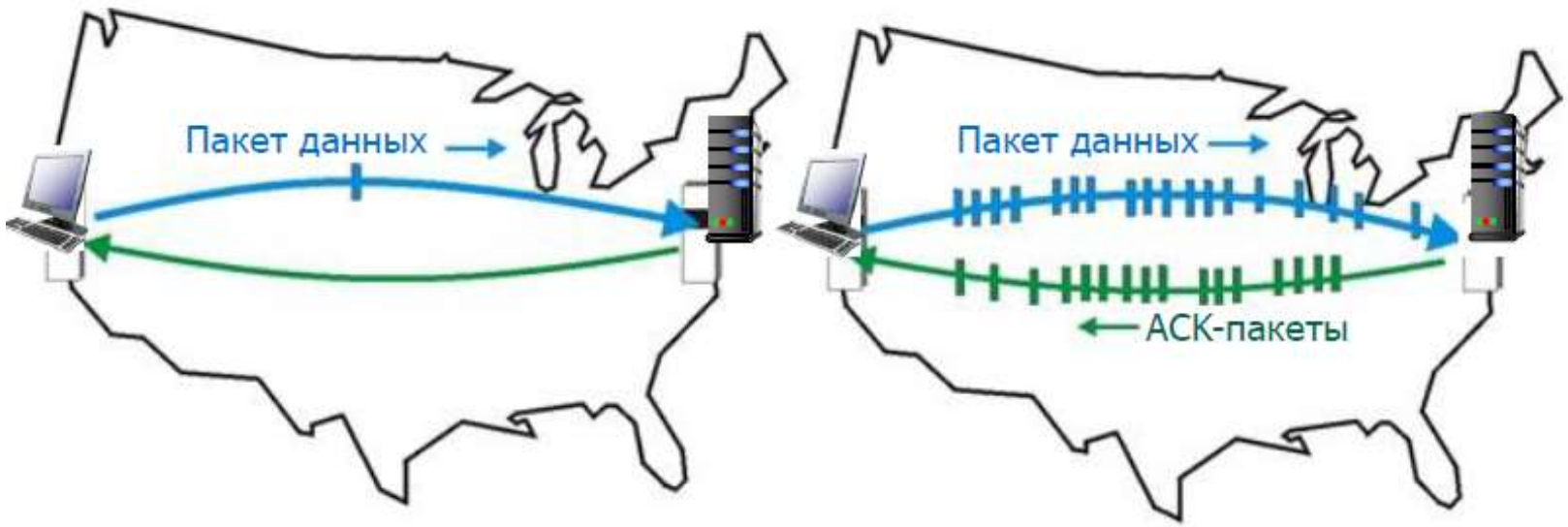


$$U_{\text{отправитель}} = \frac{L/R}{RTT + L/R} = \frac{0{,}008}{30{,}008} = 0{,}00027$$

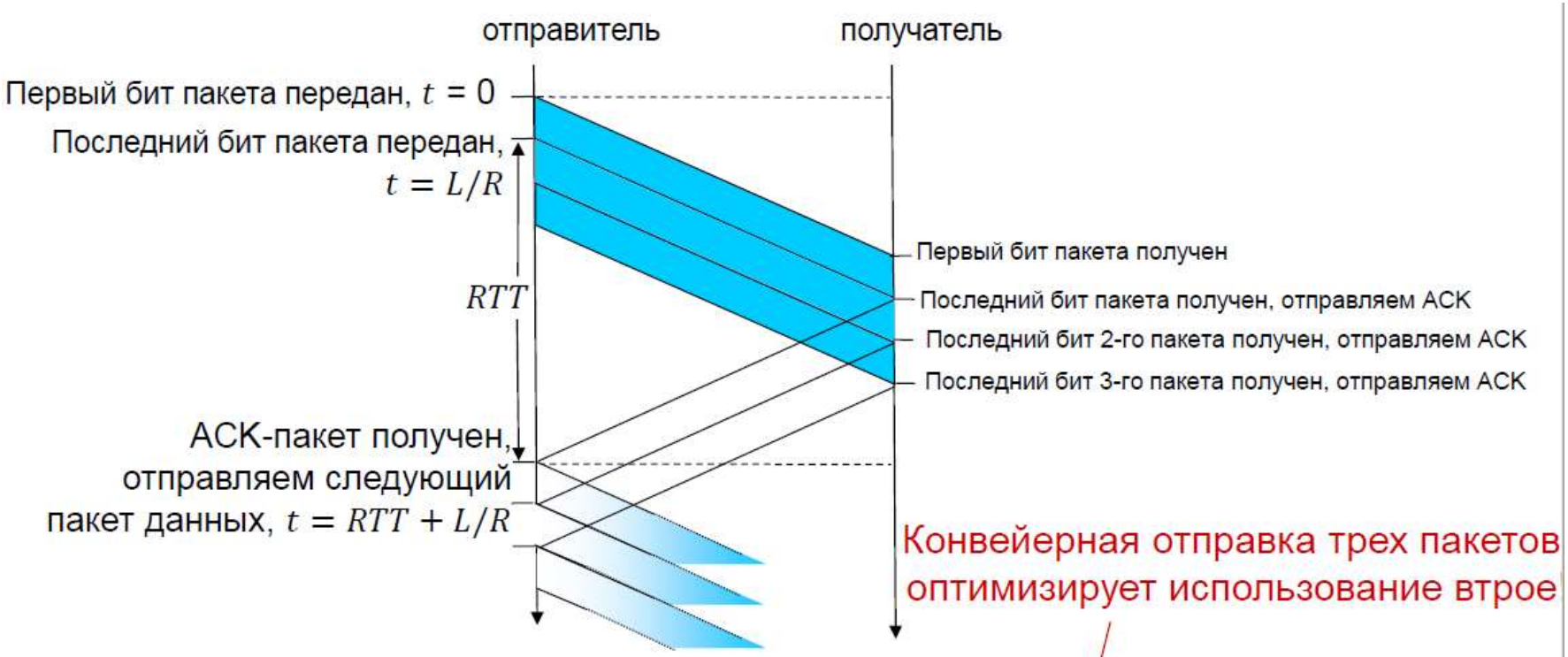
Протоколы с конвейеризацией

Конвейеризация: отправитель может одновременно отправлять несколько пакетов, не дожидаясь подтверждений

- Диапазон порядковых номеров должен быть увеличен
- Буферизация у отправителя и/или получателя
- Два базовых вида протокола с конвейеризацией: на N шагов назад и выборочное повторение



Конвейеризация: повышение использования канала



Конвейерная отправка трех пакетов оптимизирует использование втрое

$$U_{\text{отправитель}} = \frac{3L/R}{RTT + L/R} = \frac{0{,}0024}{30{,}008} = 0{,}00081$$

Протоколы с конвейеризацией: обзор

Возврат N шагов назад:

- Отправитель может одновременно предавать до N неподтвержденных пакетов
- Получатель отправляет только общий ACK-пакет
  - Не подтверждает пакеты в случае разрыва соединения
- Отправитель использует таймер наиболее раннего неподтвержденного пакета
  - Когда таймер истекает, повторно передаются все пакеты

Выборочное повторение:

- Отправитель может одновременно передавать до N неподтвержденных пакетов
- Получатель отправляет отдельные ACK-пакеты для каждого пакета
- Отправитель использует таймер для каждого неподтвержденного пакета
  - Когда таймер истекает, повторно передаются только неподтвержденные пакеты